Parte 6. Diseño y pruebas

Guía de estudio del sistema Triverde · Grupo 26 · Ingeniería de Software

Contenido

- 1. La trazabilidad de punta a punta
- 2. Patrón de diseño: Frontera, Control y Entidad
- 3. Diagramas de secuencia
- 4. Pruebas
- 5. Calidad según ISO/IEC 25010:2023
- 6. Preguntas de repaso

Esta parte cubre cómo se diseñó cada caso de uso antes de programarlo (vistas, controladores y diagramas de secuencia) y cómo se verificó después (casos de prueba, regresión, integración, cobertura y calidad ISO/IEC 25010). Corresponde a los capítulos 8 y 10 de los Word de cada incremento.

1. La trazabilidad de punta a punta

La rúbrica descuenta por cada falta de trazabilidad. El proyecto mantiene una cadena sin cortes desde lo que pidió el cliente hasta la prueba que demuestra que funciona:

```
UR-08 → RF-23 → CU-29 → V_RegistroRecepcion + C_Recepcion → secuencia cu29 normal y excepción
      → código (recepcion/serializers.py, derivar_peso_y_chip) → pruebas del servidor → caso de
prueba CP
```

Eslabón	Dónde se ve
UR → RF → CU	Matriz de trazabilidad del Documento 0 (capítulo 8)
CU → Vista y Controlador	Catálogo de vistas y controladores (`docs/referencia/Vistas y controladores.pdf`)
CU → Diagramas de secuencia	Capítulo 8 de cada Word: antes de cada bloque de secuencias hay una tabla CU → figuras → controlador → tablas físicas
CU → Código	Cada servicio, controlador y prueba cita su CU en el comentario o en la traza
CU → Caso de prueba	Tabla de trazabilidad CU → CP en el capítulo 10 de cada Word
PB → CU	Product Backlog: cada ítem dice qué CU cubre (Parte 7)

2. Patrón de diseño: Frontera, Control y Entidad

Las secuencias siguen el patrón **Boundary-Control-Entity** (BCE):

Elemento	Prefijo	Qué es	En el código
Frontera	`V_`	Pantalla con la que interactúa el actor	Componente React en `frontend/src/pages/`
Control	`C_`	Orquesta el CU: recibe, valida, coordina, responde	Uno o más ViewSets y su servicio
Entidad	`:tabla`	Objeto persistente	Tabla física en PostgreSQL

Flujo siempre igual: `Actor → V_ → C_ → :tabla`. La vista nunca toca las tablas directamente y el controlador nunca dibuja pantalla. El catálogo tiene **26 controladores** (uno por agregado o servicio, no uno por módulo) y unas 55 vistas para los 92 CU.

Los CU del actor **Sistema** no tienen vista (CU-29, 44, 46, 69, 70, 84, 90): en sus secuencias el disparo viene de otro controlador, de un evento o de una revisión periódica.

3. Diagramas de secuencia

3.1 Convenciones

- **Uno de flujo normal y uno de excepción por cada CU del incremento en curso.** No se rehacen los de incrementos anteriores: Inc 1 = 58, Inc 2 = 86, Inc 3 = 40, **184 en total**.
- Los objetos llevan el **nombre de su tabla física** (`:inventario_pila`, no `:Pila`). Fue la observación 4 del profesor en la revisión del Inc 2 y se corrigió hacia atrás en todos.
- Los controladores usan el **nombre canónico del catálogo** (`C_Trazabilidad`, `C_Reportes`, `C_Documentos`). Cuando una especificación usó otro nombre, se alineó al catálogo en el código y en los diagramas.
- Se escriben en **PlantUML** en ASCII puro (sin tildes ni ñ) y se renderizan a PNG y SVG con `java -jar tools/plantuml.jar`. Están en `docs/trabajo-grupal/diagramas/secuencias/` del repositorio.

3.2 Cómo leer uno

Ejemplo: CU-37, registrando el triturado, flujo normal (Figura 6.1).

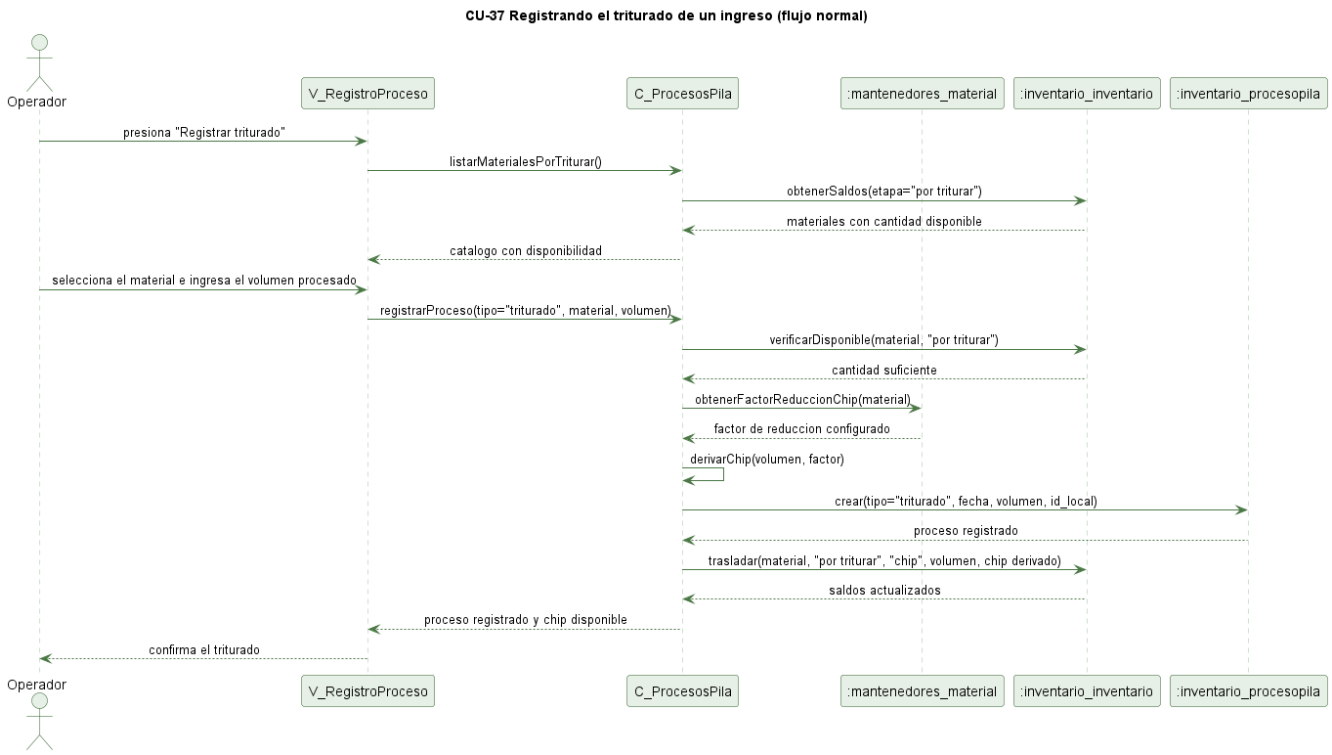


Figura 6.1. Secuencia del CU-37, flujo normal

1. El **Operador** pulsa “Registrar triturado” en `V_RegistroProceso`.
2. La vista pide a `C_ProcesosPila` los materiales con saldo; el controlador consulta `:inventario_inventario` en la etapa `por triturar`.
3. El operador elige material y volumen; la vista llama a `registrarProceso(tipo="triturado", ...)`.
4. El controlador verifica el disponible en `:inventario_inventario`, lee el factor de reducción en `:mantenedores_material` y **deriva el chip él mismo** (mensaje a sí mismo `derivarChip`).
5. Crea el proceso en `:inventario_procesopila` y traslada el saldo de `por triturar` a `chip`.
6. Responde a la vista, que confirma al operador.

La secuencia de **excepción** muestra el caso alternativo, por ejemplo volumen mayor al disponible o material sin factor configurado.

Elementos de notación:

Elemento	Significado
Flecha llena `->`	Mensaje o llamada
Flecha punteada `-->`	Respuesta
Mensaje a sí mismo	Cálculo interno del controlador
`alt` / `else`	Bifurcación: un camino u otro según una condición
`opt`	Bloque que ocurre solo si se cumple la condición
`loop`	Repetición, por ejemplo por cada documento al revisar vencimientos (CU-90)

3.3 Diseño y construcción

Las secuencias se diseñaron **antes** de programar (diseño primero) y después se contrastaron con el código. Si al construir cambió algo, se corrigió la secuencia para que coincida con lo implementado. Ejemplo: la secuencia normal del CU-68 se extendió hasta `:inventario_aporterecepcionpila` → `:recepcion_detallerecepcion` → `:recepcion_recepcion` cuando se agregó la tabla de aportes.

4. Pruebas

4.1 Niveles

Nivel	Qué prueba	Herramienta	Resultado (corte 28-09)
Servidor (unitarias e integración)	Modelos, servicios, API, permisos y excepciones de los 92 CU, con peticiones HTTP reales y la base real	`unittest` de Django sobre PostgreSQL	**263 aprobadas** en 17,5 s
Regresión	Todo lo anterior se vuelve a correr en cada incremento	La misma suite	Inc 1: 30 · Inc 2: 121 · Inc 3: 112
Extremo a extremo (integración)	Navegador real contra el sistema completo, revisando pantalla, llamada a la API y traza del servidor	Playwright (Chromium)	**38 escenarios aprobados** en 42,9 s
Cobertura	Porcentaje de sentencias ejecutadas por las pruebas	coverage.py	**92,4 %**
Calidad	Características de ISO/IEC 25010:2023	Tabla de evaluación	Cumple, con observaciones

Comando: desde `backend/`, `docker compose exec -T web python manage.py test --verbosity=2`. Cada prueba aparece con su nombre y "ok" en el registro. Los registros están en `docs/assets/pruebas/`.

4.2 Pruebas por módulo

Módulo	Pruebas	Módulo	Pruebas
M1 Acceso	9	M7 Proyecciones	9
M2 Configuración	14	M8 Comercial	47
M3 Mantenedores	3	M9 Trazabilidad	34
M4 Recepción	16	M10 Reportes	19
M5 Inventario	49	M11 Mantenimiento	15
M6 Mezcla	8	M12 Documental	40

4.3 El caso de prueba documentado

Cada prueba automatizada se documenta como caso de prueba con esta tabla: ID (CP1, CP2, CP3 según el incremento, CPI para integración), caso de uso asociado (y su excepción), objetivo, precondiciones, datos

de entrada, pasos, resultado esperado, **resultado obtenido** (que cita el nombre de la prueba y su “ok” en el registro) y estado.

Ejemplo: **CP3-05**, CU-65. Objetivo: la misma descarga no duplica el folio. Resultado obtenido: aprobado, con el nombre de la prueba automatizada en el registro. Así cada caso de prueba tiene **evidencia** verificable, no solo una afirmación.

Cada CU tiene pruebas de su flujo normal **y de sus excepciones**, y de los permisos por rol (por ejemplo, que el operador reciba 403 en los certificados).

4.4 Las pruebas de extremo a extremo

Archivos	Incremento	Escenarios	CU
`01-cotizacion` a `05-permisos`	Inc 2	19	CU-55 a CU-64 y permisos del comercial
`06-certificados`	Inc 3	5	CU-65, CU-66
`07-sinader`	Inc 3	2	CU-67
`08-trazabilidad-lote`	Inc 3	2	CU-68
`09-reportes-panel`	Inc 3	5	CU-72, 73, 76, 77
`10-documentos`	Inc 3	5	CU-86, 87, 88, 91

Están en `frontend/e2e/`. Cada escenario abre el navegador, completa el formulario, revisa lo que muestra la pantalla y lee el registro de trazas del servidor (`logs/trazas-cu.log`) para comprobar que el cálculo ocurrió en el servidor. Ejemplo: en la cotización, la prueba exige encontrar `CU-55 cotizacion.calculada`.

4.5 Cobertura

92,4 % de las sentencias del servidor, sin contar migraciones ni archivos de prueba. Las apps del Inc 3 están entre 92 % y 100 % por archivo. Los archivos con menor cobertura son de incrementos anteriores: `mantenimiento/views.py` (71 %), `mantenedores/views.py` (72 %) y `proyecciones/services.py` (76 %). Si se pregunta qué no está cubierto, esa es la respuesta honesta.

5. Calidad según ISO/IEC 25010:2023

Característica	Qué se revisó	Evidencia	Resultado
Adecuación funcional	Que los CU estén implementados y se comporten como dicen	92 de 92 CU en el servidor, 263 pruebas aprobadas	Cumple
Eficiencia de desempeño	Tiempo de respuesta	Suite en 17,5 s; 38 escenarios con navegador en 42,9 s	Cumple para el volumen de la planta
Compatibilidad	Contrato estable entre partes	Las Vistas usan solo la API REST en JSON	Cumple; falta probar Safari en iPhone
Capacidad de interacción	Mensajes claros, menú por rol	Errores de la API traducidos a mensajes; menú filtrado por rol	Cumple

Característica	Qué se revisó	Evidencia	Resultado
Fiabilidad	No perder datos si falla la conexión	Cola local y sincronización idempotente	Cumple
Seguridad	Autenticación, permisos, registro	Token, pruebas de rechazo por rol en cada módulo, bitácora	Cumple
Mantenibilidad	Cobertura y separación	92,4 %, una app por módulo, capas separadas	Cumple
Flexibilidad (portabilidad)	Instalación igual en distintos equipos	Docker Compose con el documento de instalación	Cumple
Protección	Riesgos para personas o equipos	El sistema no controla maquinaria	No aplica

Nota: el objetivo específico OE11 de la asignatura menciona “ISO 25001”; la evaluación de calidad del producto se hizo con la familia ISO/IEC 25000, en particular la 25010, que es la que define las características de calidad.

6. Preguntas de repaso

¿Qué es el patrón BCE? Frontera (la vista), Control (el controlador que orquesta el CU) y Entidad (el objeto persistente). En las secuencias son `V_`, `C_` y `:tabla`.

¿Cuántos diagramas de secuencia hay y por qué? 184: uno normal y uno de excepción por cada CU, hechos en el incremento en que se construyó cada CU.

¿Por qué los objetos de las secuencias se llaman como tablas? Porque el profesor pidió que coincidan con la base real. El nombre sale de Django y es `app_modelo`.

¿Qué diferencia hay entre `alt`, `opt` y `loop`? `alt` elige entre caminos, `opt` ocurre solo si se cumple una condición, `loop` se repite.

¿Qué tipos de prueba tienen? Del servidor (263, contra PostgreSQL real), de regresión (la suite completa en cada incremento), de extremo a extremo con navegador (38) y la evaluación de calidad ISO/IEC 25010.

¿Qué es una prueba de regresión? Volver a correr todas las pruebas anteriores para comprobar que lo nuevo no rompió lo que ya funcionaba.

¿Cómo demuestran que un caso de prueba se ejecutó? El resultado obtenido cita el nombre de la prueba automatizada, que aparece con “ok” en el registro de ejecución adjunto.

¿Qué cobertura tienen y qué es lo menos cubierto? 92,4 %. Lo más bajo son las vistas de mantenimiento y mantenedores y el servicio de proyecciones, todos de incrementos anteriores.

¿Qué prueban las pruebas de extremo a extremo que no prueban las del servidor? Que la pantalla funciona en un navegador real, que llama a la API correcta y, mediante las trazas, que el cálculo lo hizo el servidor.

¿Cómo se sigue la trazabilidad de un CU hasta su prueba? CU → vista y controlador del catálogo → secuencias del capítulo 8 → código que cita el CU → casos de prueba del capítulo 10 con su evidencia.

¿Qué observación de calidad queda abierta? Probar la compatibilidad en Safari de iPhone; las pruebas de extremo a extremo se corrieron en Chromium.